

VVS Assignment 2

fc54484 - Wenye Cheng

1 Introduction

This report presents the testing strategy and implementation for the Calendar application, a Spring Boot-based web system for managing meetings and discovering external events.

The objective of this assignment is to demonstrate different levels of software testing, including:

- Unit tests for the Business Logic with mocks
- Integration tests
 - For 3rd party sources
 - At the REST API level
 - Using the concrete database
- End-to-End tests using Selenium and a testing database
- Continuous Integration setup

2 Unit Tests for Business Logic

The unit tests focus on validating the core business logic of the application in isolation. All external dependencies such as repositories and encoders are mocked using Mockito, ensuring that only the behavior of the service layer is tested.

The main classes under tests are **MeetingService** and **UserService**. These services contain the core business rules of the system, such as meeting creation, invitation handling, event copying, and user registration.

The unit tests follow the Arrange-Act-Assert pattern, where test data and mocked dependencies are first prepared, the method under test is then executed, and finally the results or exceptions are verified.

Mockito is used to mock dependencies, including:

- **UserRepository**
- **PasswordEncoder**
- **MeetingRepository**
- **MeetingParticipantRepository**

2.1 MeetingService Tests

The `MeetingServiceTest` covers the following business rules:

- Invalid meeting time validation
`endBeforeStart()`: A meeting cannot be created if the end time is before the start time. The system correctly throws an `IllegalArgumentException`.
- Successful meeting creation
`newMeeting()`: A meeting is created successfully when valid data is provided, and the organizer is automatically added as an accepted participant.
- Non-existent invitee handling
`inviteeInexisted()`: If an invited user does not exist, an exception is thrown and the meeting is not saved.
- Duplicate invitee handling
`repetitiveInvitee()`: Repeated usernames in the invitation list are ignored to avoid duplicate participants.
- Invitation response logic
 - `acceptInvite()`: Accepting an invitation correctly updates participant status to `ACCEPTED`.
 - `pendingResponse()`: Responding with an invalid status such as `PENDING` throws an exception.
 - `inviteNotFound()`: Responding to a non-existent invitation throws an exception.
- Copying external events
 - `copyDiscoveredEvent()`: External discovered events can be copied into the calendar.
 - `defaultNoEndEvent()`: If the event has no end time, a default duration of 2 hours is applied.
- iCal token functionality
 - `icalToken()`: The system retrieves a user's calendar using a secure iCal token.
 - `invalidIcalToken()`: Invalid tokens correctly trigger an exception.

2.2 UserService Tests

The `UserServiceTest` validates user management logic:

- User registration
`newRegistration()`: A new user can be successfully registered. Passwords are encoded using `PasswordEncoder`.
- Duplicate username prevention
`userExisted()`: Attempting to register an existing username throws an exception.
- User retrieval
 - `getUser()`: Existing users can be retrieved by username.
 - `userInexisted()`: Non-existent users result in an exception.

3 Integration Tests for Third Party Sources

This section describes the integration tests for external event providers used in the application. The tested provider are:

- AgendaLx (`agendaLxTest()`): The test verifies that the provider can retrieve events and return a valid list of `DiscoveredEvent` objects. If events are returned, important fields such as title, source and start date are checked.
- SeatGeek (`seatGeekWithoutKey()` / `seatGeekWithKey()`): Two situations were tested. First, without an API key, the provider should return an empty list. Second, with a valid API key, the provider should successfully perform a search and return a non-null result.
- Ticketmaster (`seatGeekWithoutKey()` / `seatGeekWithKey()`): Two situations were tested. First, without an API key, the provider should return an empty list. Second, with a valid API key, the provider should successfully retrieve events.

4 Integration Tests at the REST API Level

REST API integration tests were implemented using Spring Boot Test and MockMvc.

iCal Endpoint

`icalTest()`: The `/ical/token.ics` endpoint was tested by creating a user and requesting the iCal file. The test verifies that the response is successful and contains calendar data.

Meeting Response Endpoint

`respondTest()`: The `/meetings/id/respond` endpoint was tested by creating a meeting invitation and sending an accept request. The test verifies that the participant status is updated from `PENDING` to `ACCEPTED`.

Copy Event Endpoint

`copyTest()`: The `/discover/copy` endpoint was tested by sending event information through a `POST` request. The test verifies that the request is processed successfully.

5 Integration Tests with a Concrete Database

Integration tests were implemented using a real database instead of mocked repositories.

Save and Search Data

`saveSearchTest()`: A user and a meeting were saved into the database. The test then queried the meetings and verified that the saved meeting could be retrieved correctly.

Meeting Service Integration

`meetingServiceTest()`: The `MeetingService` was tested together with the database. A meeting was created through the service layer and then retrieved from the database to verify that it was persisted successfully.

Overlapping Meetings

`overlappingTest()`: The repository method for detecting overlapping meetings was tested. A meeting was stored in the database and a query with an overlapping time period was executed. The test verified that the meeting was correctly returned.

6 End-to-End Tests using Selenium

End-to-end tests were implemented using Selenium WebDriver and a testing database. These tests simulate real user interactions with the application through a web browser.

Login

The test `login()` verifies that a pre-registered user can successfully log in and be redirected to the main page.

Create Meeting

The test `createMeeting()` verifies that a new user can create a meeting successfully:

1. Register a new user.
2. Log in with the created account.
3. Verify that the user is redirected to the calendar page.
4. Create a new meeting by filling in the form.
5. Submit the meeting and verify that the application redirects back to the calendar page.

Invite and Accept

The test `inviteAndAccept()` simulates two pre-defined users interacting:

1. Tester1 logs in.
2. Creates meeting and invites Tester2.
3. Tester1 logs out.
4. Tester2 logs in.
5. Tester2 accepts invitation.
6. Verify meeting appears in calendar.

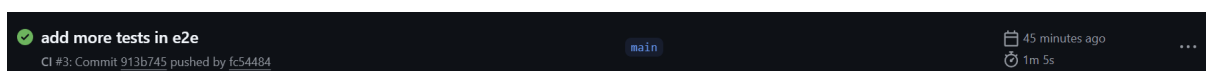
The test uses a headless Chrome browser and a Spring Boot test server running on a random port. This configuration allows the test to run automatically in both local and CI environments.

7 Continuous Integration

CI was implemented using GitHub Actions. The workflow runs automatically on every push and pull request. It builds the project and runs all tests using Maven.

The source code repository is available at: <https://github.com/fc54484/calendar.git>

The figure below shows the CI execution results on GitHub Actions.



8 Conclusion

This report described the testing approach for the Calendar application, including unit tests, integration tests, and end-to-end tests.

Each level of testing was used to check different parts of the system and ensure that the application works correctly.

Overall, the tests improve the reliability of the application.